Application layer

Functionality of Application Layer

1) Network Virtual terminal

- The application layer is the software version of a physical terminal and this layer permitted to a user to log on to a remote host.
- For this, an application creates a software emulation of a terminal at the remote host. By this user's computer can communicate with the software terminal, which in turn, communicates with the host.
- It is shown that the remote host is communicating with one of its terminals, so it allows the user to log on.

2) File Transfer, Access, and Management (FTAM)

- An application permits a user to access files in a remote computer, to retrieve files from a computer and to manage files on a remote computer.
- FTAM is concerned with a hierarchical virtual file in terms of file attributes, file structure and the types of operations performed on the files and their attributes.

3) Addressing

- To achieve communication between client and server system, there is a need for addressing.
- When a request is sent from the client side to the server side, this request contains the server address and its own address.
- The server answered to the client request, this request contains the destination address, i.e., client address. DNS is used to achieve this type of addressing.

4) Mail Services

Email forwarding and storage of e-mails provided by an application layer.

5) Directory Services

A distributed database is contained by an application that provides access for global information about various objects and services.

6) Authentication

It provides authentication to occur between devices for an extra layer of security and it authenticates the sender or receiver's message or both.

Domain Name System

- The client/server programs can be divided into two categories:
  - **those that can be directly used by the user, such as e-mail, and**
  - **those that support other application programs.**
- The Domain Name System (DNS) is a supporting program that is used by other programs such as e-mail.
- Figure shows an example of how a DNS client/server program can support an e-mail program to find the IP address of an e-mail recipient.
- A user of an e-mail program may know the e-mail address of the recipient; however, the IP protocol needs the IP address.
- The DNS client program sends a request to a DNS server to map the e-mail address to the corresponding IP address.

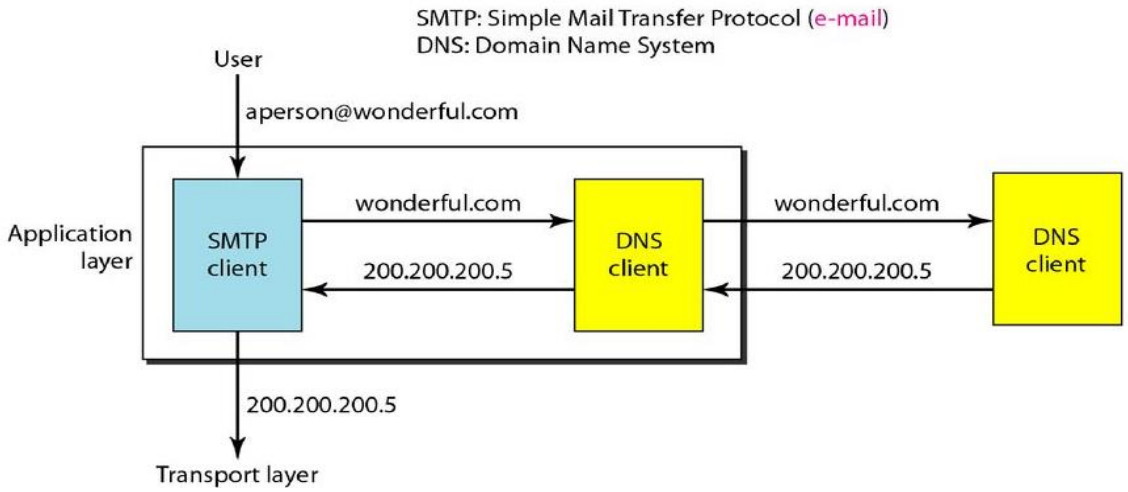

Figure: Example of using the DNS service

NAME SPACE

- The names must be unique because the addresses are unique.

- A name space that maps each address to a unique name can be organized in two ways: **flat** or **hierarchical**.

Flat Name Space

- In a flat name space, a name is assigned to an address.
- A name in this space is a sequence of characters without structure.
- The names may or may not have a common section; if they do, it has no meaning.
- The main disadvantage of a flat name space is that it cannot be used in a large system such as the Internet because it must be centrally controlled to avoid ambiguity and duplication.

Hierarchical Name Space

- In a hierarchical name space, each name is made of several parts.
- The first part can define the nature of the organization, the second part can define the name of an organization, and the third part can define departments in the organization, and so on.
- In this case, the authority to assign and control the name spaces can be decentralized.
- A central authority can assign the part of the name that defines the nature of the organization and the name of the organization.
- The responsibility of the rest of the name can be given to the organization itself.
- The organization can add suffixes (or prefixes) to the name to define its host or resources.
- For example, assume two colleges and a company call one of their computers *challenger*.
- The first college is given a name by the central authority such as *jhda.edu*, the second college is given the name *berkeley.edu*, and the company is given the name *smart. com.*
- When these organizations add the name *challenger* to the name they have already been given, the end result is three distinguishable names: *challenger.jhda.edu*, *challenger.berkeley.edu*, and *challenger.smart.com.*
- The names are unique without the need for assignment by a central authority.
- The central authority controls only part of the name, not the whole.

DOMAIN NAME SPACE

- To have a hierarchical name space, a domain name space was designed.

- In this design the names are defined in an inverted-tree structure with the root at the top.
- The tree can have only 128 levels: level 0 (root) to level 127.

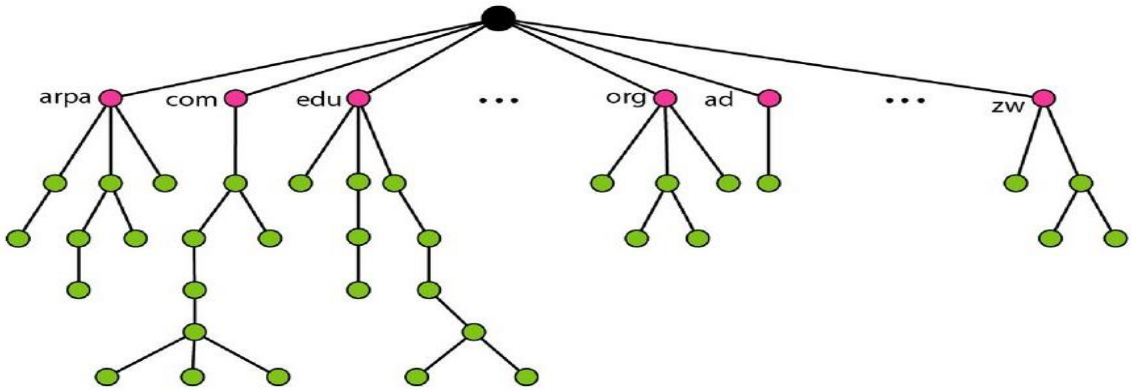

Figure: Domain name space

Label

- Each node in the tree has a label, which is a string with a maximum of 63 characters.
- The root label is a null string (empty string).
- DNS requires that children of a node (nodes that branch from the same node) have different labels, which guarantees the uniqueness of the domain names.

Domain Name

- Each node in the tree has a domain name.
- A full domain name is a sequence of labels separated by dots (.).
- The domain names are always read from the node up to the root.
- The last label is the label of the root (null). This means that a full domain name always ends in a null label, which means the last character is a dot because the null string is nothing. Figure 25.3 shows some domain names.

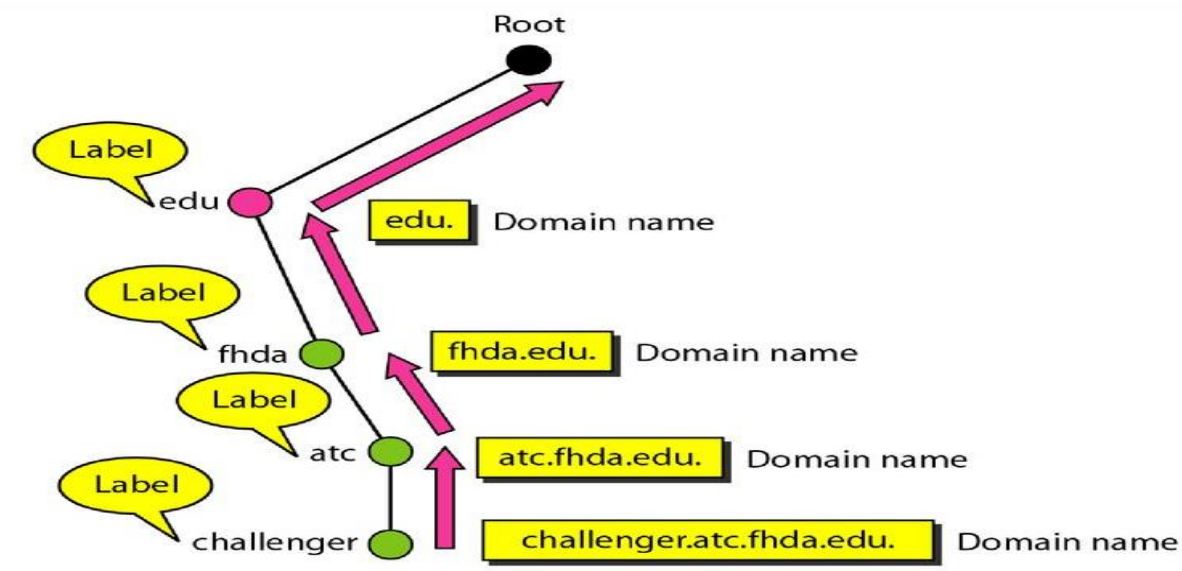

Figure: Domain names and labels

Fully Qualified Domain Name

- **If a label is terminated by a null string, it is called a fully qualified domain name (FQDN)**.
- An FQDN is a domain name that contains the full name of a host.
- It contains all labels, from the most specific to the most general, that uniquely define the name of the host.
- For example, the domain name *challenger.ate.tbda.edu.* is the FQDN of a computer named *challenger* installed at the Advanced Technology Centre (ATC) at De Anza College.
- A DNS server can only match an FQDN to an address.
- Note that the name must end with a null label, but because null means nothing, the label ends with a dot (.).

Partially Qualified Domain Name

- **If a label is not terminated by a null string, it is called a partially qualified domain name (PQDN).**
- A PQDN starts from a node, but it does not reach the root.
- It is used when the name to be resolved belongs to the same site as the client.
- Here the resolver can supply the missing part, called the suffix, to create an FQDN.
- For example, if a user at the *jhda.edu.* site wants to get the IP address of the challenger computer, he or she can define the partial name challenger.

- The DNS client adds the suffix *atc.jhda.edu.* before passing the address to the DNS server.
- The DNS client normally holds a list of suffixes. The following can be the list of suffixes at De Anza College.
- The null suffix defines nothing. This suffix is added when the user defines an FQDN.

atc.fhda.edu

fhda.edu

null

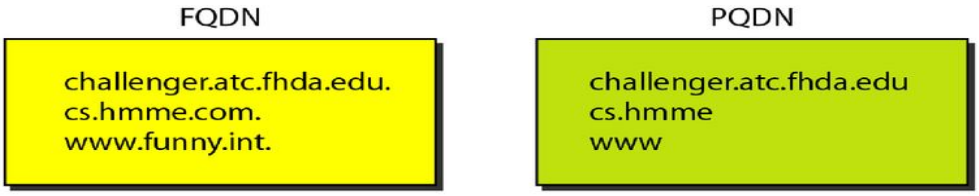

Figure: FQDN and PQDN

Domain

- A **domain** is a subtree of the domain name space.
- The name of the domain is the domain name of the node at the top of the subtree.
- Note that a domain may itself be divided into domains (or **subdomains** as they are sometimes called).

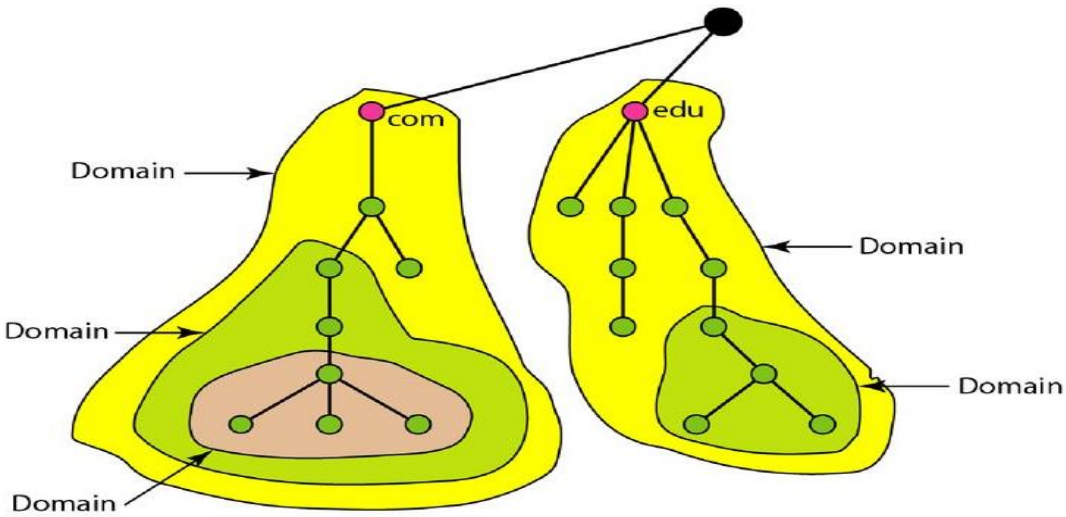

Figure: Domains

DISTRIBUTION OF NAME SPACE

- The information contained in the domain name space must be stored.
- However, it is very inefficient and also unreliable to have just one computer store such a huge amount of information.
- It is inefficient because responding to requests from all over the world places a heavy load on the system.
- It is not unreliable because any failure makes the data inaccessible.

Hierarchy of Name Servers

- The solution to these problems is to distribute the information among many computers called DNS servers.
- One way to do this is to divide the whole space into many domains based on the first level.
- In other words, we let the root stand alone and create as many domains (subtrees) as there are first-level nodes.
- Because a domain created in this way could be very large, DNS allows domains to be divided further into smaller domains (subdomains). Each server can be responsible (authoritative) for either a large or a small domain. In other words, we have a hierarchy of servers in the same way that we have a hierarchy of names.

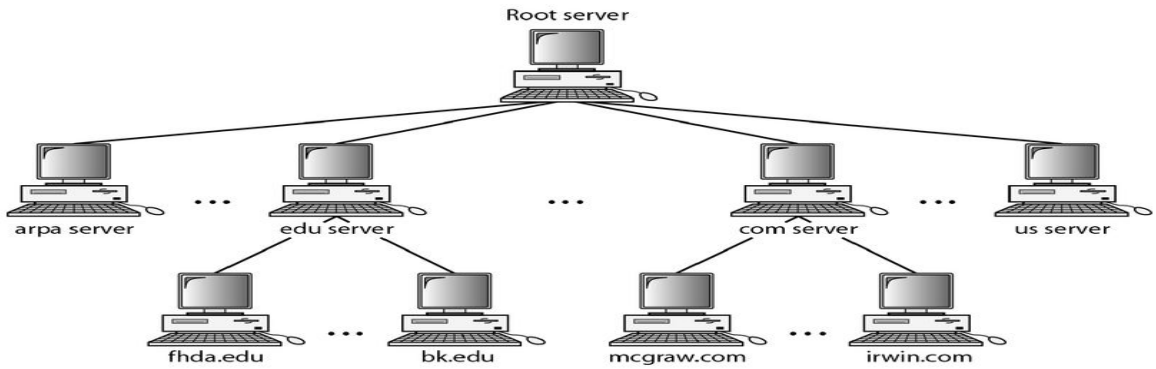

Figure: Hierarchy of name servers

Zone

- Since the complete domain name hierarchy cannot be stored on a single server, it is divided among many servers.

- What a server is responsible for or has authority over is called a zone.
- We can define a **zone as a contiguous part of the entire tree**.
- If a server accepts responsibility for a domain and does not divide the domain into smaller domains, the *domain* and the *zone* refer to the same thing.
- The server makes a database called a *zone file* and keeps all the information for every node under that domain.
- However, if a server divides its domain into subdomains and delegates part of its authority to other servers, *domain* and *zone* refer to different things.
- The information about the nodes in the subdomains is stored in the servers at the lower levels, with the original server keeping some sort of reference to these lower-level servers.
- Of course the original server does not free itself from responsibility totally: It still has a zone, but the detailed information is kept by the lower-level servers.
- A server can also divide part of its domain and delegate responsibility but still keep part of the domain for itself. In this case, its zone is made of detailed information for the part of the domain that is not delegated and references to those parts that are delegated.

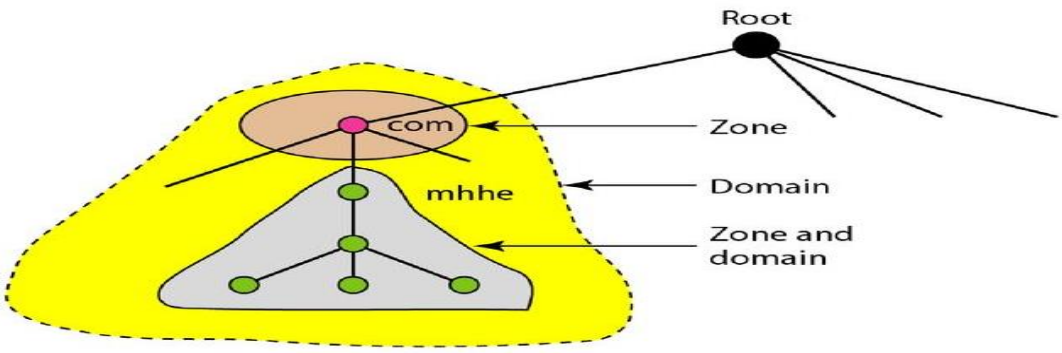

Figure: Zones and domains

Root Server

- A root server is a server whose zone consists of the whole tree.
- A root server usually does not store any information about domains but delegates its authority to other servers, keeping references to those servers.
- There are several root servers, each covering the whole domain name space. The servers are distributed all around the world.

Primary and Secondary Servers

- DNS defines two types of servers: primary and secondary.
- A primary server is a server that stores a file about the zone for which it is an authority.
- It is responsible for creating, maintaining, and updating the zone file.
- It stores the zone file on a local disk.
- A secondary server is a server that transfers the complete information about a zone from another server (primary or secondary) and stores the file on its local disk.
- The secondary server neither creates nor updates the zone files.
- If updating is required, it must be done by the primary server, which sends the updated version to the secondary.
- The primary and secondary servers are both authoritative for the zones they serve.
- The idea is not to put the secondary server at a lower level of authority but to create redundancy for the data so that if one server fails, the other can continue serving clients.
- Note also that a server can be a primary server for a specific zone and a secondary server for another zone.
- Therefore, when we refer to a server as a primary or secondary server, we should be careful to which zone we refer.
- A primary server loads all information from the disk file; the secondary server loads all information from the primary server.
- When the secondary downloads information from the primary, it is called zone transfer.

DNS IN THE INTERNET

- DNS is a protocol that can be used in different platforms.
- In the Internet, the domain name space (tree) is divided into three different sections: generic domains, country domains, and the inverse domain.

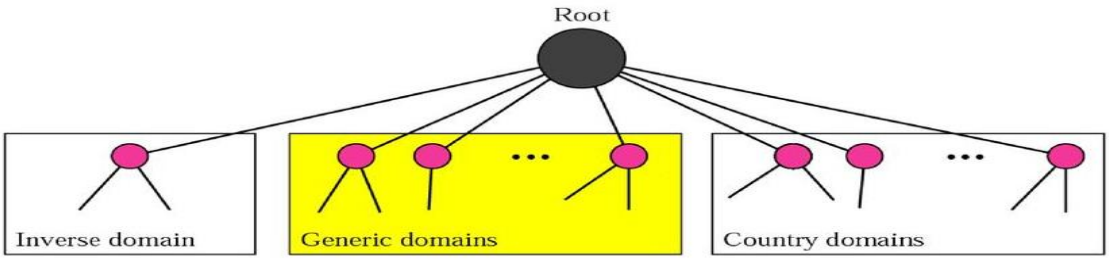

Figure: DNS used in the Internet

Generic Domains

- The **generic domains** define registered hosts according to their generic behaviour.
- Each node in the tree defines a domain, which is an index to the domain name space database.

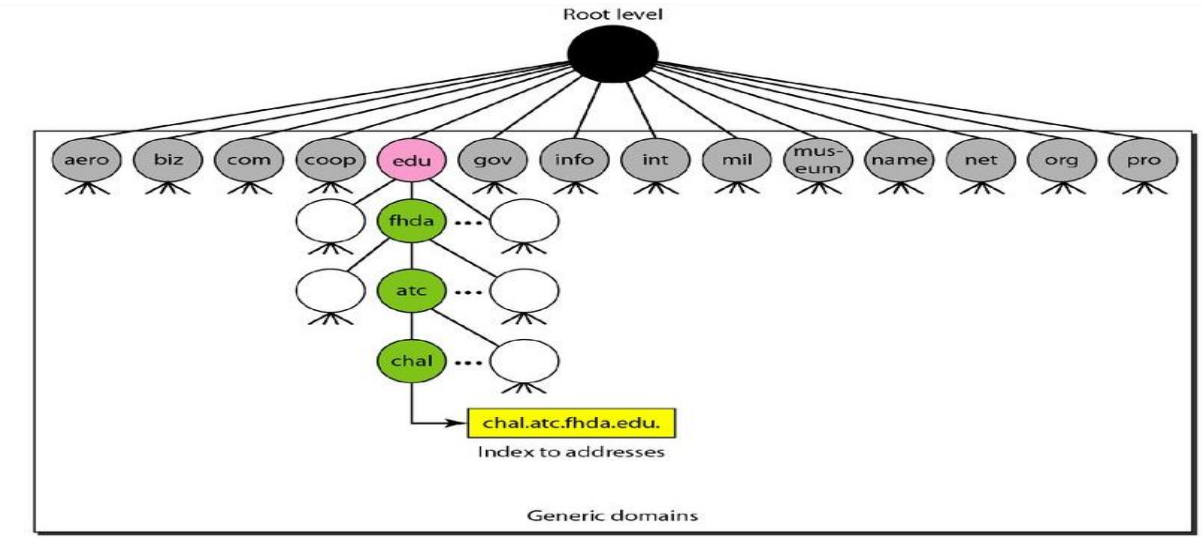

Figure: Generic domains

- Looking at the tree, we see that the first level in the generic domains section allows 14 possible labels.
- These labels describe the organization types as listed in Table.

Label	*Description*
aero	Airlines and aerospace companies
biz	Businesses or firms (similar to "com")
com	Commercial organizations
coop	Cooperative business organizations
edu	Educational institutions
gov	Government institutions
info	Information service providers
int	International organizations
mil	Military groups
museum	Museums and other nonprofit organizations
name	Personal names (individuals)
net	Network support centers
org	Nonprofit organizations
pro	Professional individual organizations

Table: Generic domain labels

Country Domains

- The country domains section uses two-character country abbreviations (e.g., us for United States).
- Second labels can be organizational, or they can be more specific, national designations.
- The United States, for example, uses state abbreviations as a subdivision of us (e.g., ca.us.).
- Figure shows the country domains section. The address *anza.cup.ca.us* can be translated to De Anza College in Cupertino, California, in the United States.

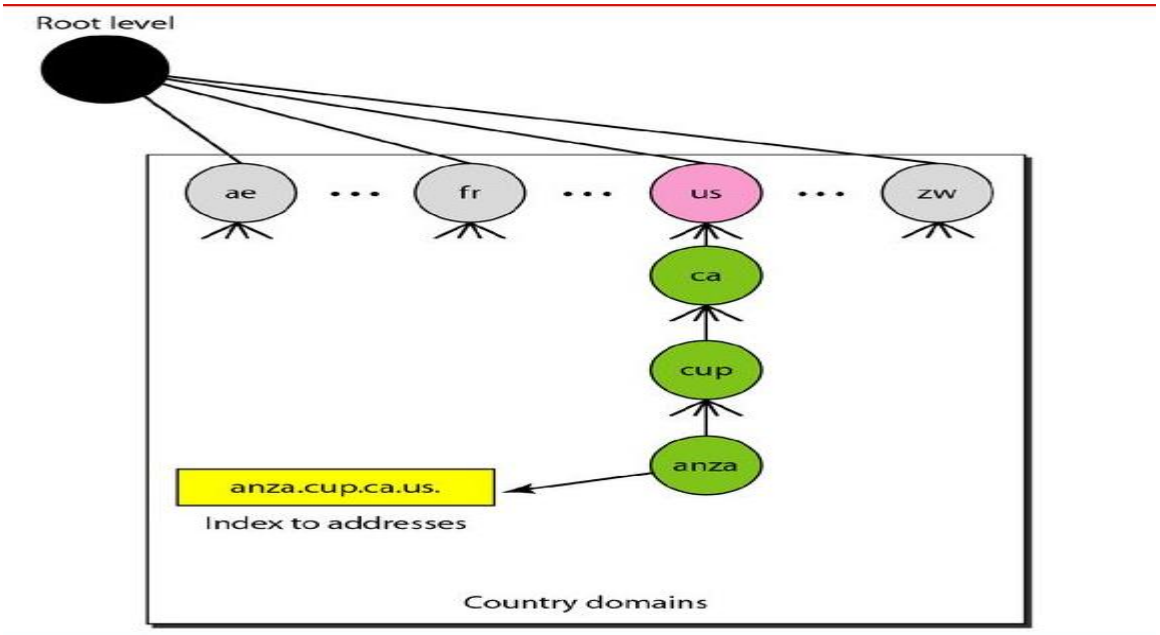

Figure: Country domains

Inverse Domain

- The inverse domain is used to map an address to a name.
- This may happen, for example, when a server has received a request from a client to do a task.
- Although the server has a file that contains a list of authorized clients, only the IP address of the client (extracted from the received IP packet) is listed.
- The server asks its resolver to send a query to the DNS server to map an address to a name to determine if the client is on the authorized list.
- This type of query is called an inverse or pointer (PTR) query.

- To handle a pointer query, the inverse domain is added to the domain name space with the first-level node called *arpa* (for historical reasons).
- The second level is also one single node named *in-addr* (for inverse address).
- The rest of the domain defines IP addresses.
- The servers that handle the inverse domain are also hierarchical.
- This means the netid part of the address should be at a higher level than the subnetid part, and the subnetid part higher than the hostid part.
- In this way, a server serving the whole site is at a higher level than the servers serving each subnet.
- This configuration makes the domain look inverted when compared to a generic or country domain.
- To follow the convention of reading the domain labels from the bottom to the top, an IF address such as 132.34.45.121 (a class B address with netid 132.34) is read as 121.45.34.132.in-addr. arpa.

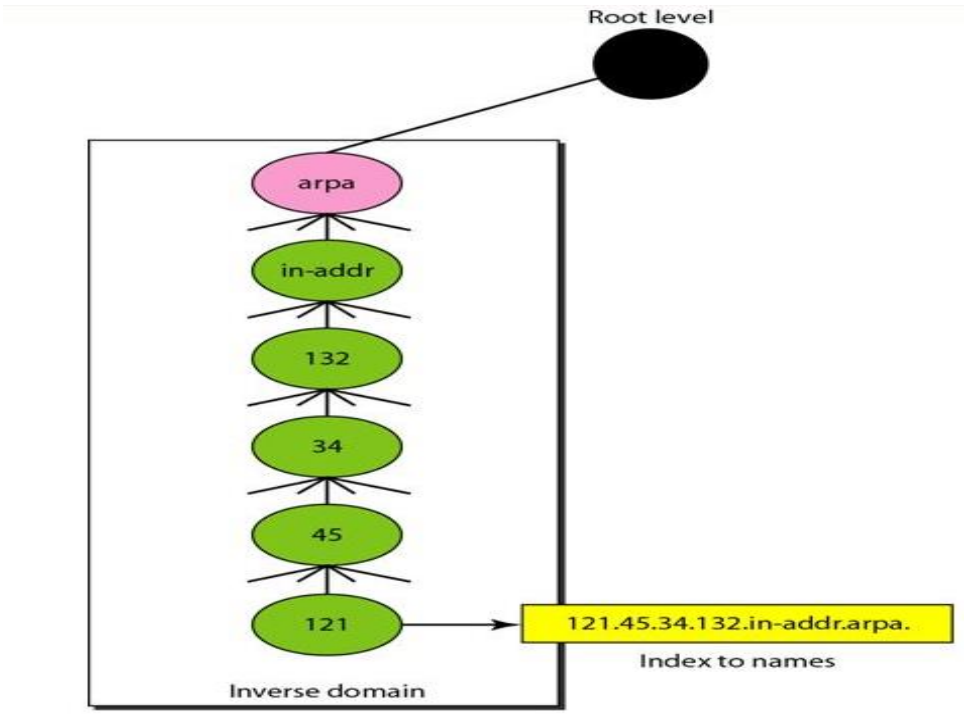

Figure: Inverse domain

RESOLUTION

- Mapping a name to an address or an address to a name is called *name-address resolution.*

Resolver

- DNS is designed as a client/server application.
- A host that needs to map an address to a name or a name to an address calls a DNS client called a resolver.
- The resolver accesses the closest DNS server with a mapping request.
- If the server has the information, it satisfies the resolver; otherwise, it either refers the resolver to other servers or asks other servers to provide the information.
- After the resolver receives the mapping, it interprets the response to see if it is a real resolution or an error, and finally delivers the result to the process that requested it.

Mapping Names to Addresses

- Most of the time, the resolver gives a domain name to the server and asks for the corresponding address.
- **In** this case, the server checks the generic domains or the country domains to find the mapping.
- If the domain name is from the generic domains section, the resolver receives a domain name such as *"chal.atc.jhda.edu."*.
- The query is sent by the resolver to the local DNS server for resolution.
- **If** the local server cannot resolve the query, it either refers the resolver to other servers or asks other servers directly.
- If the domain name is from the country domains section, the resolver receives a domain name such as *"ch.jhda.cu.ca.us."*. The procedure is the same.

Mapping Addresses to Names

- A client can send an **IP** address to a server to be mapped to a domain name. As mentioned before, this is called a PTR query.
- To answer queries of this kind, DNS uses the inverse domain. However, in the request, the **IP** address is reversed and the two labels *in-addr* and *arpa* are appended to create a domain acceptable by the inverse domain section.
- For example, if the resolver receives the IF address 132.34.45.121, the resolver first inverts the address and then adds the two labels before sending.
- The domain name sent is *"121.45.34.132.in-addr.arpa."* which is received by the local DNS and resolved.

Recursive Resolution

- The client (resolver) can ask for a recursive answer from a name server. This means that the resolver expects the server to supply the final answer.
- If the server is the authority for the domain name, it checks its database and responds.
- If the server is not the authority, it sends the request to another server (the parent usually) and waits for the response.
- If the parent is the authority, it responds; otherwise, it sends the query to yet another server.
- When the query is finally resolved, the response travels back until it finally reaches the requesting client. This is called recursive resolution and is shown in Figure.

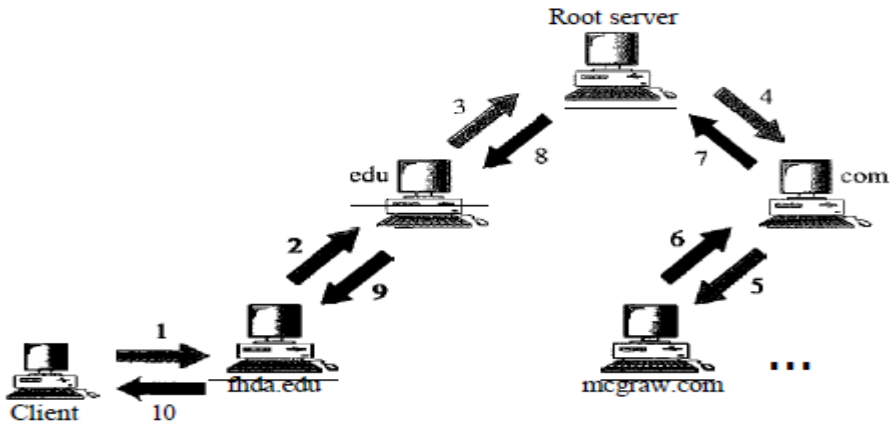

Figure: Recursive resolution

Iterative Resolution

- If the client does not ask for a recursive answer, the mapping can be done iteratively.
- If the server is an authority for the name, it sends the answer. If it is not, it returns (to the client) the IP address of the server that it thinks can resolve the query.
- The client is responsible for repeating the query to this second server.
- If the newly addressed server can resolve the problem, it answers the query with the IP address; otherwise, it returns the IP address of a new server to the client.
- Now the client must repeat the query to the third server. This process is called iterative resolution because the client repeats the same query to multiple servers.
- In Figure the client queries four servers before it gets an answer from the mcgraw.com server.

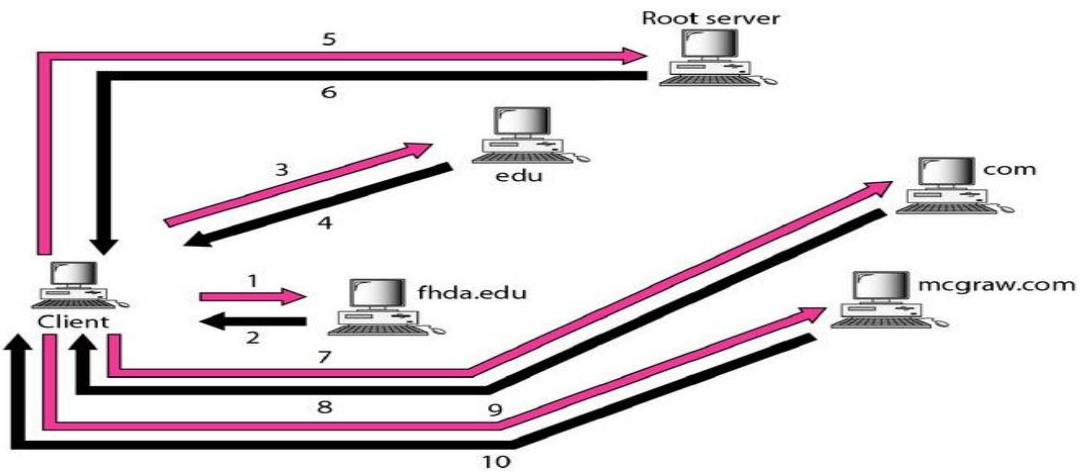

Figure: Iterative resolution

Caching

- Each time a server receives a query for a name that is not in its domain, it needs to search its database for a server IP address. Reduction of this search time would increase efficiency.
- DNS handles this with a mechanism called caching.
- When a server asks for a mapping from another server and receives the response, it stores this information in its cache memory before sending it to the client.
- If the same or another client asks for the same mapping, it can check its cache memory and solve the problem.
- However, to inform the client that the response is coming from the cache memory and not from an authoritative source, the server marks the response as *unauthoritative.* Caching speeds up resolution, but it can also be problematic.
- If a server caches a mapping for a long time, it may send an outdated mapping to the client. To counter this, two techniques are used.
- First, the authoritative server always adds information to the mapping called *time-to-live* (TTL).
- It defines the time in seconds that the receiving server can cache the information.
- After that time, the mapping is invalid and any query must be sent again to the authoritative server.
- Second, DNS requires that each server keep a TTL counter for each mapping it caches.
- The cache memory must be searched periodically, and those mappings with an expired TTL must be purged.